

100 Days of Azure DevOps

Day 79 Handout — Write the runbook before 2am, and write the review without blame

8 - Monitoring & Observability · Personal lab only · Educational content · Not a sales pitch

Architecture A — the incident loop (lab-sized)

Stage	Output you write	Not this
Detect	Alert / health URL / user report timestamp	Waiting for a feeling that 'it seems down'
Checks	Ordered probes: DNS, App Service state, App Insights failures	A 12-page novel nobody opens
Mitigate	Swap slot, scale, revert pipeline, disable a flag	Root-cause essay while users are still down
Communicate	Who gets a status line, what is true now	Silent debug while rumors fill the gap
Postmortem	What happened, what we believed, what changes	Who is stupid — that sentence trains silence

Architecture B — mitigate first, then learn

Situation	Do now	Do after users work
App Service HTTP 5xx	Restart / swap staging slot / last good artifact	Dump Kudu and write a novel
Bad deploy 10 min ago	Redeploy previous Build.BuildId	Hotfix on main with no revert path
Secret leaked in a run	Rotate in Key Vault; fail the pipeline	Leave the value in the log 'until Monday'
You do not know yet	Mitigate with the last known-good + say 'investigating'	Invent a root cause to look senior
Lab has no real outage	Still write the page — 2am brains need headings	Skip the template because 'nothing broke'

Architecture C — pitfalls that book a sequel outage

Pitfall	What you see	Fix
Blame review	Quieter chat, same outage later	Facts, beliefs, changes — no names as villains
Novel runbook	Unread wiki during the incident	One page: Symptom → Checks → Mitigate → Comms → PM
Mitigate = RCA	Users down while you 'find the why'	Stop the bleeding first
No comms step	People invent a status	Write who gets a message in the runbook
Employer incident paste	You almost retell a work outage	Fictional lab app only. No workplace detail

One-liner to remember

Postmortems without blame create learning; postmortems with blame create silence.

Step-by-step lab (personal Azure / Azure DevOps only)

Personal repo azure-100-labs. Fictional lab app only — no employer incident details.

- 1 Create docs/runbook-webapp-down.md with headings: Symptom → Checks → Mitigate → Communicate → Postmortem link.
- 2 Fill Checks with concrete probes: Portal App Service status, <https://<app>.azurewebsites.net/health>, App Insights failures, last pipeline deploy.
- 3 Fill Mitigate with last-good artifact / slot swap / stop-start — not a root-cause paragraph.

- 4 Add docs/postmortem-template.md: Summary, Timeline, What we believed, What was true, What we change, Follow-ups. No blame section.
- 5 If you had a real lab failure this week, fill the template once. If not, leave headings plus one fictional timeline labeled FICTION.
- 6 Link the two files from README or docs/index. A page that cannot be found at 2am does not exist.

Runbook skeleton — short enough for a 2am brain

```
# docs/runbook-webapp-down.md
# Symptom: GET / or /health != 200 for >2 min
# Checks:
#   az webapp show -g rg-azure100-lab -n app-azure100-lab --query state
#   curl -sI https://<app>.azurewebsites.net/health
# Mitigate: az webapp deployment slot swap -g ... -n ... --slot staging
#           or redeploy artifact $(Build.BuildId) from the last green run
# Communicate: status line to yourself + anyone using the lab URL
# Postmortem: docs/postmortem-template.md
```

Done checklist

- docs/runbook-webapp-down.md has the five headings and at least one real CLI/URL check
- docs/postmortem-template.md is blameless (no 'who failed' section)
- Mitigate is separate from root cause
- No employer outage story
- Posted the LinkedIn document

Tomorrow — Day 80

Phase 8 recap — App Insights + one alert + one dashboard, and a forced failure you can see in five minutes